

Module: Auth

[pawguard/modules/auth](#) — API prefix: `/admin`, `/auth`

PawGuard Backend | FastAPI 3.13 + PostgreSQL 17 + Redis/ARQ + S3 | Generated from repo @ commit 99a8430 (2026-08-12)

1. Purpose

The platform's identity and access core: registration, login, RS256 JWT session issuance, refresh/session management, RBAC/PBAC permission checks, MFA (TOTP), OAuth account linking, and the audit-event log all other modules write to.

2. Position in the Layered Architecture

[Client](#) → [Router \(auth+validate\)](#) → [Service \(business logic\)](#) → [Repository \(data access\)](#) → [PostgreSQL](#)

Like every module in this codebase, business rules live only in the Service layer; Routers authenticate/authorize/validate and Repositories perform data access with no business decisions (see AGENTS.md — the project's engineering constitution).

3. Data Model — Owned Tables

Model Class	Table Name
Role	roles
Permission	permissions
RolePermission	role_permissions
UserRole	user_roles
User	users
UserSession	user_sessions
RefreshToken	refresh_tokens
MFADevice	mfa_devices
PasswordResetToken	password_reset_tokens
EmailVerificationToken	email_verification_tokens
OAuthAccount	oauth_accounts
AuthAuditLog	auth_audit_logs

4. API Endpoints (33 total)

Method	Path	Permission Required
GET	/admin/roles	system:admin
POST	/admin/roles	system:admin
GET	/admin/roles/{role_id}	system:admin
PUT	/admin/roles/{role_id}	system:admin
DELETE	/admin/roles/{role_id}	system:admin
GET	/admin/permissions	system:admin
GET	/admin/users	system:admin
POST	/admin/users	system:admin
GET	/admin/users/{user_id}	system:admin
PUT	/admin/users/{user_id}	system:admin
DELETE	/admin/users/{user_id}	system:admin

Method	Path	Permission Required
POST	/admin/register	—
POST	/admin/login	—
POST	/admin/mfa/verify	—
POST	/admin/refresh	—
POST	/admin/logout	—
POST	/admin/logout-all	—
GET	/admin/me	—
PUT	/admin/me	—
GET	/admin/sessions	—

+13 additional endpoints not shown — see `openapi.json` for the full list.

Distinct permission codes used: `system:admin`

5. Key Service-Layer Operations

- `change_password`
- `confirm_email_verification`
- `confirm_mfa_enrollment`
- `confirm_password_reset`
- `create_role`
- `create_user`
- `delete_role`
- `delete_user`
- `disable_mfa`
- `enroll_mfa`
- `get_role`
- `get_user`
- `link_oauth_account`
- `list_oauth_accounts`

+18 additional service methods.

6. Notable Implementation Details

- Argon2id password hashing; RS256 JWT access tokens (15-min) + opaque SHA-256 refresh tokens.
- Redis-cached RBAC/PBAC permission lookups (5-min TTL); TOTP MFA secrets encrypted at rest with Fernet.
- OAuth (Google/Apple) login fails closed if the configured client ID is empty — audience is always verified.